Homework 06 - Labeling legislative bills

INFO 4940/5940 - Fall 2025

Nick Johnson

2025-11-19

Exercise 1

```
# scripts/leg-label.py

import pandas as pd
from dotenv import load_dotenv
from pydantic import BaseModel
import pyarrow.feather as feather
import chatlas as ctl
from chatlas import batch_chat_structured

# ---------- Env + paths ----------
load_dotenv()

INPUT_FEATHER = "data/leg_lite.feather"
OUTPUT_CSV = "data/llm_predictions.csv"

PROMPT_SIMPLE = "prompts/cap-simple.md"
PROMPT_DETAILED = "prompts/cap-detailed.md"
PROMPT_REASONING = "prompts/cap-reasoning.md"

# 🔬 Testing controls
TEST_ONLY = False
TEST_N = 3

# Map “labels” used in your analysis to actual model IDs
MODELS = {
    "gpt-4.1": "gpt-4.1",
    "gpt-5-nano": "gpt-5-nano",
    "gpt-5": "gpt-5",
}
```

```
PROMPTS = {
    "naive": PROMPT_SIMPLE,
    "detailed": PROMPT_DETAILED,
    "reasoning": PROMPT_REASONING,
}


def load_prompt(path: str) -> str:
    """Load prompt from a markdown/text file."""
    with open(path, encoding="utf-8") as f:
        return f.read()


# ---------- Structured output model ----------

class PolicyPrediction(BaseModel):
    """Structured output for CAP classification."""
    policy_number: int
    reasoning: str | None = None


def main():
    # ---------- Load data ----------
    print(f"Loading data from {INPUT_FEATHER} ...")
    df_full = feather.read_feather(INPUT_FEATHER)

    expected_cols = {"description", "policy"}
    missing = expected_cols - set(df_full.columns)
    if missing:
        raise ValueError(f"Missing expected column(s) in leg_lite.feather:
{missing}")

    df_full = df_full.reset_index(drop=True).copy()
    df_full["row_id"] = df_full.index
    df_full["policy"] = df_full["policy"].astype(int)

    if TEST_ONLY:
        df = df_full.iloc[:TEST_N].copy()
        print(f"TEST_ONLY is True → running on first {len(df)} rows")
    else:
        df = df_full
        print(f"TEST_ONLY is False → running on all {len(df)} rows")

    all_results: list[dict] = []

    # ---------- Loop over model × prompt, use batch_chat_structured
----------
    for model_label, model_id in MODELS.items():
```

```
        for prompt_label, prompt_path in PROMPTS.items():
            print(f"\n=== Running batch for model={model_label}
prompt={prompt_label} ===")

            system_prompt = load_prompt(prompt_path)

            # Chat client for this combo
            chat = ctl.ChatOpenAI(
                model=model_id,
                system_prompt=system_prompt,
            )

            # List of descriptions for this batch
            prompts = df["description"].tolist()

            # Path to store batch state/results
            batch_state_path = f"data/batch-{model_label}-{prompt_label}.json"

            print(f"Submitting batch job → {batch_state_path}")

            # ✅ Correct signature: no `convert=` argument
            predictions = batch_chat_structured(
                chat=chat,
                prompts=prompts,
                path=batch_state_path,
                data_model=PolicyPrediction,
                wait=True,   # block until results are ready
            )

            # `predictions` is a list of PolicyPrediction or None for failures
            if len(predictions) != len(df):
                raise RuntimeError(
                    f"Expected {len(df)} predictions, got {len(predictions)}"
                )

            # Attach predictions back to rows
            for row, pred in zip(df.itertuples(index=False), predictions):
                if pred is None:
                    # If a row failed, you can decide how to handle it; here
we use -1
                    pred_label = -1
                    reasoning = "Batch request failed for this row."
                else:
                    pred_label = int(pred.policy_number)
                    reasoning = pred.reasoning or ""

                all_results.append(
                    {
```

```
                        "row_id": int(row.row_id),
                        "description": row.description,
                        "true_label": int(row.policy),
                        "model_label": model_label,
                        "model_id": model_id,
                        "prompt_label": prompt_label,
                        "pred_label": pred_label,
                        "reasoning": reasoning,
                        # Token usage is not currently exposed by
batch_chat_structured
                        "total_tokens": None,
                        "input_tokens": None,
                        "output_tokens": None,
                    }
                )

    # ---------- Save predictions as CSV ----------
    results_df = pd.DataFrame(all_results)
    print(f"\nSaving predictions to {OUTPUT_CSV} ...")
    results_df.to_csv(OUTPUT_CSV, index=False)
    print("Done.")


if __name__ == "__main__":
    mode = "TEST" if TEST_ONLY else "FULL"
    print(f"Running scripts/leg-label.py (chatlas batch_chat_structured,
mode={mode})")
    main()
```

Naive and simple prompt

You classify U.S. congressional bill descriptions into one of the 20 major policy topics used by the Comparative Agendas Project.

Read the description and decide which single policy topic number (1–20) best matches the bill's main purpose. Focus on the primary policy area, not minor details or boilerplate phrases like "and for other purposes." If the text touches multiple issues, choose the topic that is most central to the bill's intent.

Explicit code/label values

You are an expert policy coder for the Comparative Agendas Project (CAP). Your job is to assign each U.S. congressional bill description to exactly one CAP major topic based on its primary policy focus.

Use the bill's main purpose to choose the best-fitting topic number. Ignore boilerplate language like "and for other purposes" and secondary side effects. When multiple topics appear, pick the one that is most central to what the bill is trying to accomplish.

Here are the CAP major policy topic codes:

1 – Macroeconomics
2 – Civil rights, minority issues, civil liberties
3 – Health
4 – Agriculture
5 – Labor and employment
6 – Education
7 – Environment
8 – Energy
9 – Immigration
10 – Transportation
11 – Law, crime, family issues
12 – Social welfare
13 – Community development and housing
14 – Banking, finance, domestic commerce
15 – Defense
16 – Space, technology, communications
17 – Foreign trade
18 – International affairs, foreign aid
19 – Government operations
20 – Public lands and water management

Detailed and reasoning prompt

You are an expert policy analyst trained in the Comparative Agendas Project (CAP) coding rules. Your task is to assign a U.S. congressional bill description to one CAP major policy topic (1–20) based on the bill's primary purpose.

Follow this process when you classify:

1. Read the description carefully and identify the core policy problem the bill addresses.
2. Match that problem to the closest CAP major topic, focusing on the main goal rather than minor side effects or boilerplate phrases like "and for other purposes."
3. If several topics are mentioned, choose the single topic that best captures the central intent of the bill.
4. Be prepared to briefly explain why this topic is more appropriate than nearby alternatives (for example, energy vs. environment, health vs. social welfare, etc.).

Here are the CAP major policy topic codes:

1 – Macroeconomics
2 – Civil rights, minority issues, civil liberties
3 – Health
4 – Agriculture
5 – Labor and employment
6 – Education

7 – Environment
8 – Energy
9 – Immigration
10 – Transportation
11 – Law, crime, family issues
12 – Social welfare
13 – Community development and housing
14 – Banking, finance, domestic commerce
15 – Defense
16 – Space, technology, communications
17 – Foreign trade
18 – International affairs, foreign aid
19 – Government operations
20 – Public lands and water management

Exercise 2

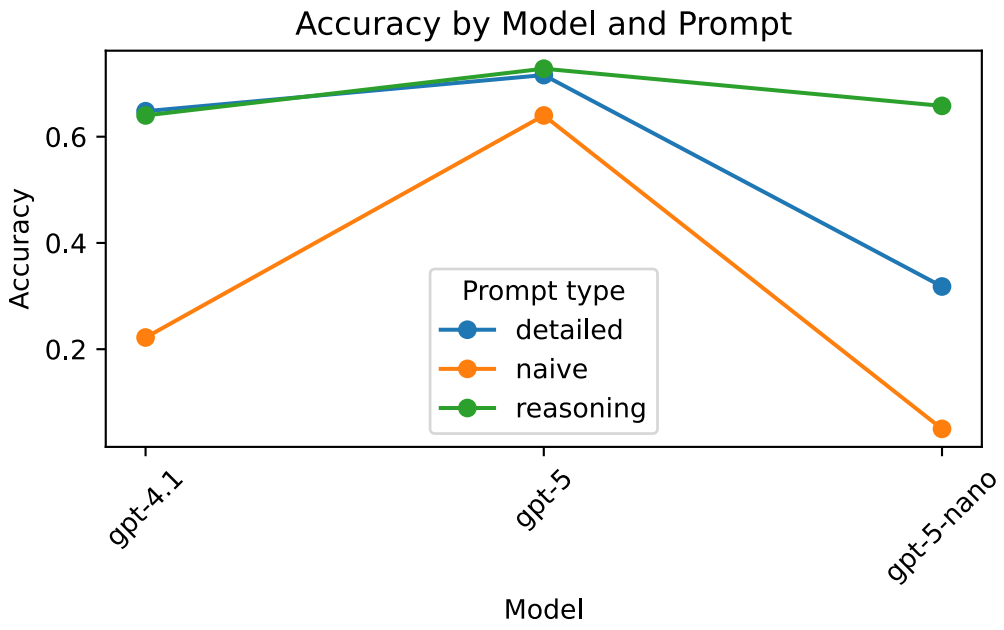
```
import pyarrow.feather as feather

import numpy as np
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix

import pandas as pd

preds = pd.read_csv("data/llm_predictions.csv")


# Make sure types are right
preds["true_label"] = preds["true_label"].astype(int)
preds["pred_label"] = preds["pred_label"].astype(int)


def multiclass_sensitivity_specificity(y_true, y_pred):
    """
    Compute macro-averaged sensitivity (recall) and specificity
    for a multiclass problem using the confusion matrix.
    """
    labels = np.unique(y_true)
    cm = confusion_matrix(y_true, y_pred, labels=labels)

    sensitivities = []
    specificities = []

    for i, _ in enumerate(labels):
        tp = cm[i, i]
        fn = cm[i, :].sum() - tp
```

```
        fp = cm[:, i].sum() - tp
        tn = cm.sum() - tp - fn - fp

        sens = tp / (tp + fn) if (tp + fn) > 0 else np.nan
        spec = tn / (tn + fp) if (tn + fp) > 0 else np.nan

        sensitivities.append(sens)
        specificities.append(spec)

    return float(np.nanmean(sensitivities)), float(np.nanmean(specificities))


rows = []

for (model, prompt), group in preds.groupby(["model_label", "prompt_label"]):
    y_true = group["true_label"].to_numpy()
    y_pred = group["pred_label"].to_numpy()

    acc = accuracy_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred, average="macro")
    sens, spec = multiclass_sensitivity_specificity(y_true, y_pred)

    rows.append(
        {
            "model": model,
            "prompt": prompt,
            "accuracy": acc,
            "f1_macro": f1,
            "sensitivity_macro": sens,
            "specificity_macro": spec,
        }
    )

metrics_df = pd.DataFrame(rows).sort_values(["model",
"prompt"]).reset_index(drop=True)
metrics_df
```

	model	prompt	accuracy	f1_macro	sensitivity_macro	specificity_macro
0	gpt-4.1	detailed	0.648	0.624376	0.619057	0.981182
1	gpt-4.1	naive	0.222	0.183037	0.197199	0.958887
2	gpt-4.1	reasoning	0.640	0.612605	0.594063	0.980736
3	gpt-5	detailed	0.716	0.667050	0.664353	0.984804
4	gpt-5	naive	0.640	0.593111	0.604821	0.980917
5	gpt-5	reasoning	0.728	0.670488	0.669870	0.985477

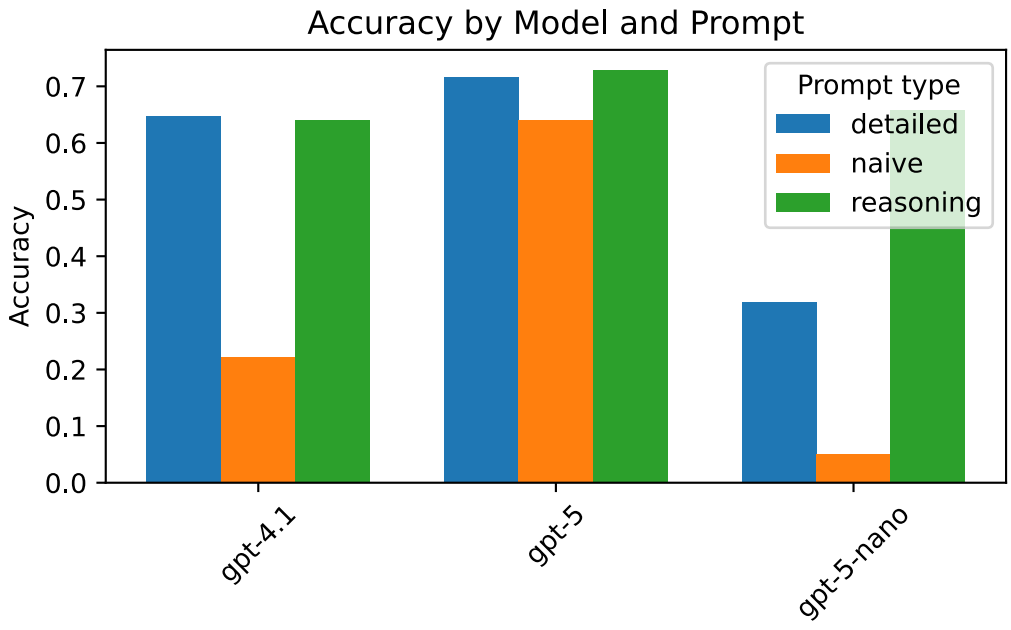
	model	prompt	accuracy	f1_macro	sensitivity_macro	specificity_macro
6	gpt-5-nano	detailed	0.318	0.345126	0.580041	0.979487
7	gpt-5-nano	naive	0.050	0.046698	0.062564	0.950094
8	gpt-5-nano	reasoning	0.658	0.596047	0.638995	0.982055

```
import matplotlib.pyplot as plt

plt.figure()
for prompt in metrics_df["prompt"].unique():
    subset = metrics_df[metrics_df["prompt"] == prompt]
    plt.plot(subset["model"], subset["accuracy"], marker="o", label=prompt)

plt.ylabel("Accuracy")
plt.xlabel("Model")
plt.title("Accuracy by Model and Prompt")
plt.legend(title="Prompt type")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



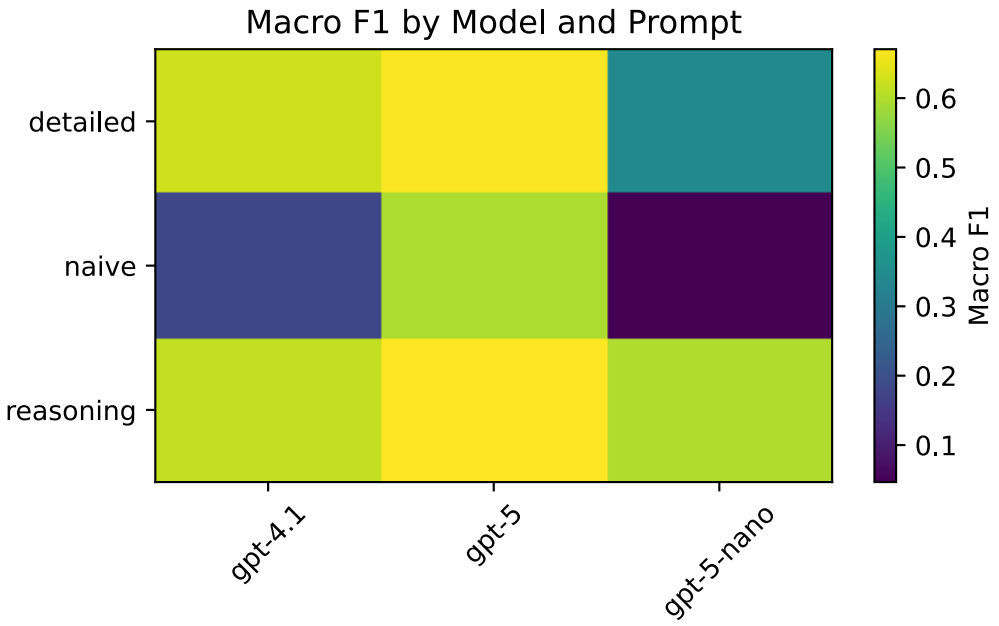

```
import matplotlib.pyplot as plt
import numpy as np

# Make sure metrics_df exists from previous chunk
```

```
models = metrics_df["model"].unique()
prompts = metrics_df["prompt"].unique()

x = np.arange(len(models))  # positions for models on x-axis
width = 0.25                # width of each bar

plt.figure()

for i, prompt in enumerate(prompts):
    subset = metrics_df[metrics_df["prompt"] == prompt]
    # Align subset to models order
    subset = subset.set_index("model").loc[models]
    plt.bar(x + i * width, subset["accuracy"], width=width, label=prompt)

plt.xticks(x + width, models, rotation=45)
plt.ylabel("Accuracy")
plt.title("Accuracy by Model and Prompt")
plt.legend(title="Prompt type")
plt.tight_layout()
plt.show()
```




```
import matplotlib.pyplot as plt

# Pivot F1 into a matrix: rows = prompt, cols = model
```

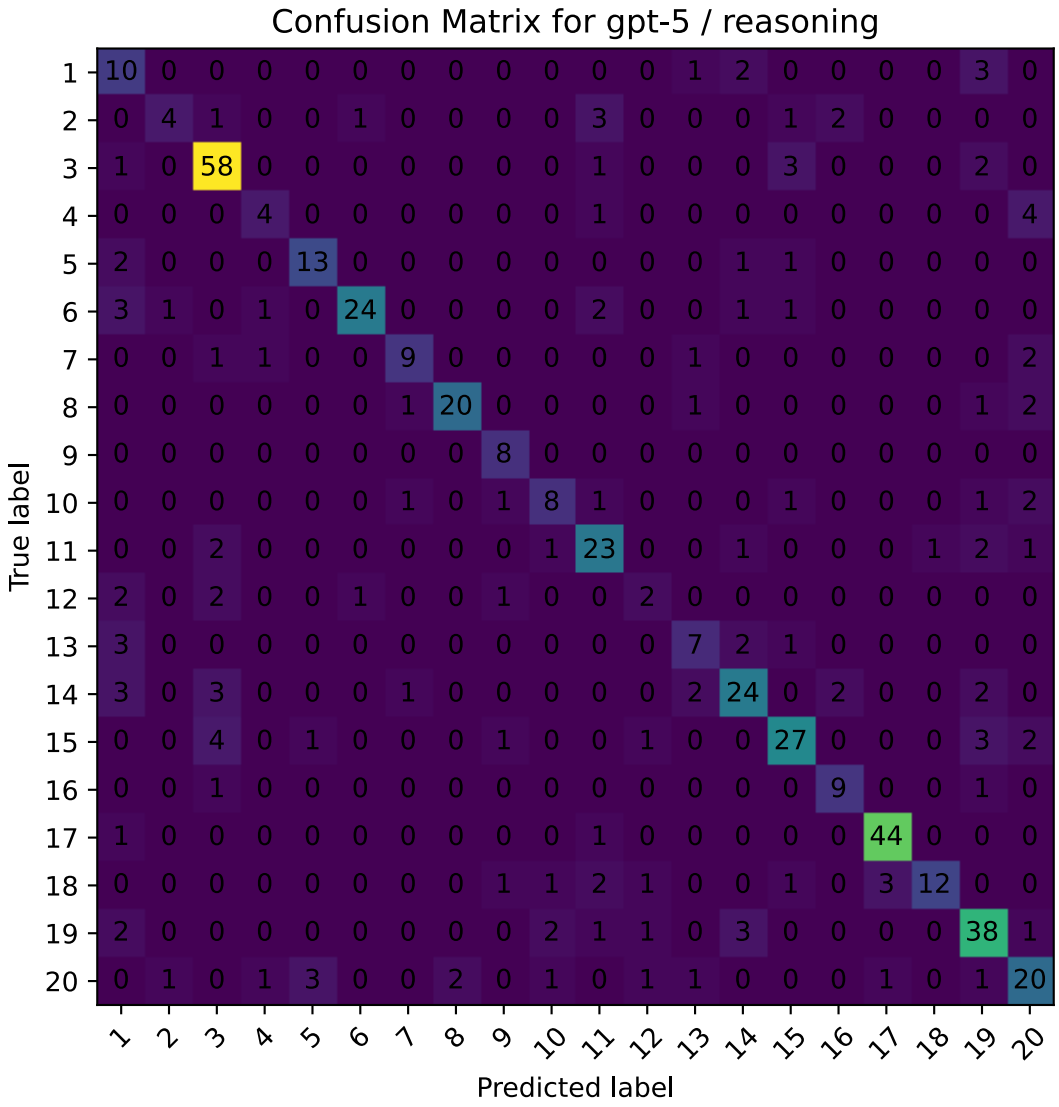
```
f1_pivot = metrics_df.pivot(index="prompt", columns="model",
values="f1_macro")

plt.figure()
plt.imshow(f1_pivot.values, aspect="auto")
plt.colorbar(label="Macro F1")

plt.xticks(range(len(f1_pivot.columns)), f1_pivot.columns, rotation=45)
plt.yticks(range(len(f1_pivot.index)), f1_pivot.index)

plt.title("Macro F1 by Model and Prompt")
plt.tight_layout()
plt.show()
```




```
from sklearn.metrics import confusion_matrix

# Find the best combo by accuracy (you can change to f1_macro if you prefer)
best_row = metrics_df.sort_values("accuracy", ascending=False).iloc[0]
best_model = best_row["model"]
best_prompt = best_row["prompt"]
print("Best combo:", best_model, best_prompt)

# Filter predictions for that combo

best_preds = preds[
    (preds["model_label"] == best_model) & (preds["prompt_label"] ==
```

```
best_prompt)
]

y_true = best_preds["true_label"].astype(int).to_numpy()
y_pred = best_preds["pred_label"].astype(int).to_numpy()

labels = sorted(np.unique(y_true))
cm = confusion_matrix(y_true, y_pred, labels=labels)

plt.figure(figsize=(6, 6))
plt.imshow(cm, interpolation="nearest")
plt.title(f"Confusion Matrix for {best_model} / {best_prompt}")
plt.xlabel("Predicted label")
plt.ylabel("True label")
plt.xticks(range(len(labels)), labels, rotation=45)
plt.yticks(range(len(labels)), labels)

# Add counts on the heatmap

for i in range(len(labels)):
    for j in range(len(labels)):
        plt.text(j, i, cm[i, j], ha="center", va="center")

plt.tight_layout()
plt.show()# Exercise 3
```

```
Best combo: gpt-5 reasoning
```




```
library(shiny)
library(bslib)

ui <- page_fillable()

server <- function(input, output, session) {}

shinyApp(ui, server)
```

Link to app: https://019a9d96-3e1e-232e-db76-84b5f6eba95f.share.connect.posit.cloud/

```
from __future__ import annotations
```

```python
from textwrap import dedent

from shiny import App, ui, reactive
from dotenv import load_dotenv
from chatlas import ChatOpenAI

from rag import get_top_k_similar_documents  # your RAG helper

# ---------------- Environment ---------------- #

load_dotenv()  # OPENAI_API_KEY from .env locally; from env vars on Connect

# ---------------- System Prompt ---------------- #

system_prompt = dedent("""
    You are a course tutor for INFO 4940 / 5940 at Cornell.

    Your role:
    - Answer questions about topics covered in this course (LLMs, prompts,
RAG, evaluation, dashboards, etc.).
    - Help students reason through homework and projects WITHOUT giving full
solutions.
    - Explain code and concepts clearly using R or Python examples when
requested.
    - Use the provided context (syllabus, assignment text, project
description) as your primary factual source.

    What you MUST do:
    - Be honest about what you know and what you don't.
    - Encourage good academic integrity: guide, don't solve.
    - Prefer course materials for questions about policies, grading, and due
dates.
    - When giving code, keep it fairly short and explain it.

    What you MUST NOT do:
    - Do NOT write full homework or project answers that could be submitted
as-is.
    - Do NOT fabricate course policies or due dates.
    - Do NOT provide long, copy-pasteable solutions to graded questions.

    Style:
    - Friendly, concise, and concrete.
    - If the student asks for a specific language, use that (R or Python).
    - If they don't specify, default to Python for code.
    - If their question is vague, ask one clarifying question before
answering.
""").strip()
```

```
# ---------------- LLM Client ---------------- #

chat_client = ChatOpenAI(
    model="gpt-4.1-mini",  # adjust to your allowed model
    system_prompt=system_prompt,
)

# ---------------- UI ---------------- #

app_ui = ui.page_sidebar(
    ui.sidebar(
        ui.h4("Tutor settings"),

        ui.input_radio_buttons(
            "help_mode",
            "What do you need help with?",
            choices=[
                "Concepts (lectures/readings)",
                "Assignments & projects",
                "Code debugging",
                "Course policies (syllabus)",
                "Study strategies",
            ],
            selected="Concepts (lectures/readings)",
        ),

        ui.input_radio_buttons(
            "pref_lang",
            "Preferred language for examples",
            choices=["No preference", "Python", "R"],
            selected="No preference",
        ),

        ui.hr(),
        ui.input_action_button(
            "show_tips",
            "Show tips for good questions",
            class_="btn-secondary w-100",
        ),
        ui.br(),
        ui.input_action_button(
            "show_examples",
            "Show example questions",
            class_="btn-outline-secondary w-100",
        ),
    ),
```

```
    ui.layout_columns(
        ui.card(
            ui.card_header("INFO 4940 / 5940 Tutor Bot"),
            ui.markdown(
                "Ask about **lectures**, **homework**, **projects**, "
                "**course policies**, or **how to structure your code**.\n\n"
                "The tutor uses the course syllabus and assignment PDFs as
context, "
                "and will *guide* you rather than doing your work for you."
            ),
            ui.chat_ui(
                "tutor_chat",
                placeholder="Ask your question here…",
            ),
        ),
    ),

    title="INFO 4940 / 5940 Tutor Bot",
    fillable=True,
    fillable_mobile=True,
)

# ---------------- Server ---------------- #

def server(input, output, session):
    # Chat instance for the UI chat component
    chat = ui.Chat(id="tutor_chat")

    # --- Popups (modals) ----------------------------------------------------

    tips_modal = ui.modal(
        ui.h4("Tips for using the tutor"),
        ui.tags.ul(
            ui.tags.li("Ask focused questions (e.g., one assignment or concept
at a time)."),
            ui.tags.li("Mention the assignment number or topic when
relevant."),
            ui.tags.li("Say “in R” or “in Python” if you have a preference."),
            ui.tags.li("If you're stuck on code, paste the error message and a
small code snippet."),
        ),
        title="Tips",
        easy_close=True,
        footer=ui.input_action_button(
            "close_tips",
            "Close",
            class_="btn-primary"
        ),
```

```
    )

    examples_modal = ui.modal(
        ui.h4("Example questions"),
        ui.tags.ul(
            ui.tags.li(""What is RAG and how is it different from fine-
tuning?""),
            ui.tags.li(""Can you remind me of the evaluation criteria for
Homework 4?""),
            ui.tags.li(""How should I structure a system prompt for a labeling
task?""),
            ui.tags.li(""In Python, how do I read a Feather file and compute
accuracy and F1?""),
            ui.tags.li(""What is the late policy for homework in this
class?""),
        ),
        title="Example questions",
        easy_close=True,
        footer=ui.input_action_button(
            "close_examples",
            "Close",
            class_="btn-primary"
        ),
    )

    @reactive.Effect
    @reactive.event(input.show_tips)
    def _show_tips():
        ui.modal_show(tips_modal)

    @reactive.Effect
    @reactive.event(input.close_tips)
    def _close_tips():
        ui.modal_remove()

    @reactive.Effect
    @reactive.event(input.show_examples)
    def _show_examples():
        ui.modal_show(examples_modal)

    @reactive.Effect
    @reactive.event(input.close_examples)
    def _close_examples():
        ui.modal_remove()

    # --- Chat handler ------------------------------------------------------

    @chat.on_user_submit
```

```
    async def handle_user_input(user_input: str):
        help_mode = input.help_mode()
        pref_lang = input.pref_lang()

        # 1. Retrieve relevant course docs via RAG
        top_docs = get_top_k_similar_documents(user_input, top_k=4)

        if top_docs:
            context_block = "\n\n---\n\n".join(top_docs)
        else:
            context_block = "No course documents were retrieved."

        # 2. Build full prompt
        prompt = f"""You are tutoring an INFO 4940 / 5940 student.

Student help mode: {help_mode}
Preferred language for examples: {pref_lang}

Use the context below, if relevant, to answer their question.
If the context does not answer the question, say that and answer
based on general LLM knowledge, but do not make up specific course
policies or due dates.

Context:
{context_block}

Student question:
{user_input}
"""

        # 3. Stream back to chat UI
        response_stream = await chat_client.stream_async(prompt)
        await chat.append_message_stream(response_stream)


app = App(app_ui, server)
```

Generative AI (GAI) self-reflection